第七讲 - 函数 (编程练习)

张建章

阿里巴巴商学院

杭州师范大学

2026-09



1 业务问题引入

2 从重复代码到可复用规则

3 函数的定义、调用与返回值

4 参数设计

5 作用域与数据流

6 函数的设计与测试

7 综合业务实践

8 总结与课后拓展

Python 程序设计基础 第7讲 函数 速记卡

把重复规则封装起来，让代码更清晰！



1 为什么要学函数？

- 在电商业务中，很多订单都会使用**同一套结算规则**。

订单 O001 单价：120 数量：3
订单 O002 单价：80 数量：8
订单 O003 单价：260 数量：2
.....

使用同一套结算规则

- 多笔订单会重复使用同一套结算规则
- 如果复制同样代码，规则变化时很难一起修改
- **函数** = 给一段可复用规则起名字
- 作用：减少重复、便于修改、提高可读性

100笔订单也能重复调用同一个规则！

2 从重复代码到可复用规则

把重复的结算逻辑写成一个函数，以后直接调用。

```
def calculate_payable(unit_price, quantity):
    subtotal = unit_price * quantity
    discount = 0
    if subtotal >= 500:
        discount = subtotal * 0.10
    after_discount = subtotal - discount
    shipping = 20
    if after_discount >= 400:
        shipping = 0
    return after_discount + shipping
```

调用示例：

```
>>> calculate_payable(120, 3)  → 380.0
>>> calculate_payable(80, 8)   → 576.0
>>> calculate_payable(260, 2)  → 468.0
```



规则只写一次，后面反复调用！

3 函数的定义、调用与返回值

def	开始定义函数
函数名	说明函数要完成的任务
参数	接收调用时传入的数据
return	把结果交回给调用者

```
def calculate_subtotal(unit_price, quantity):
    return unit_price * quantity
```

调用： `calculate_subtotal(120, 3)` → **返回：** 360



- 有 return：得到可继续使用的结果
- 没有 return：默认返回 None

4 参数设计

- 不同的参数方式，适用于不同的场景。

位置参数	按顺序传
关键字参数	按参数名传
默认参数	不给就用默认值




```
def calculate_shipping(payable,
                       free_shipping_threshold=400,
                       standard_fee=20):
    if payable >= free_shipping_threshold:
        return 0
    return standard_fee
```

调用示例：

```
>>> calculate_shipping(380)                                  → 20
>>> calculate_shipping(380, free_shipping_threshold=300)     → 0
```

参数越清楚，函数越容易读懂和检查。

5 作用域与数据流

- 函数内部的变量叫**局部变量**
- 局部变量只能在函数内部使用
- 数据最好**“从参数进入，从返回值出去”**

不好的做法：依赖外部变量

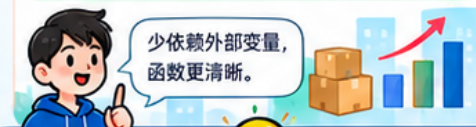
```
discount_rate = 0.10

def calculate_discount(amount):
    return amount * discount_rate
```

依赖外部变量，函数不够清晰

更好的做法：通过参数传入

```
def calculate_discount(amount, discount_rate):
    return amount * discount_rate
```



6 好函数长什么样？如何测试？

- 一个函数只做一件清楚的事
- 函数名、参数名有业务含义
- 输入靠参数，输出靠返回值
- 可以单独测试

```
def calculate_discount(amount: float,
                       discount_rate: float) -> float:
    """根据订单金额和折扣率计算优惠金额。"""
    return amount * discount_rate
```

测试示例：

```
assert calculate_shipping(399) == 20
assert calculate_shipping(400) == 0
assert calculate_shipping(380, free_shipping_threshold=300) == 0
```

普通值 + 边界值，都要测。

7 综合业务实践：订单结算

订单数据：

```
order = {"id": "O002", "unit_price": 80, "quantity": 8}
```

结算规则：

1. 商品金额 ≥ 500：打9折
2. 折后金额 ≥ 400：免运费
3. 否则运费 20 元



核心函数（分小步）：

```
def calculate_subtotal(unit_price, quantity): ...
def calculate_discount(subtotal): ...
def calculate_shipping(after_discount): ...
def settle_order(order): ...
```

结果示例：

- 商品金额：640 元
- 优惠金额：64 元
- 运费：0 元
- 实付金额：576 元

数据流：

商品金额 → 优惠金额 → 折后金额 → 运费 → 实付金额

8 学习提醒 / 易错点

- 先定义函数，再调用函数
- return 和 print 不一样：return 交结果，print 只是显示
- 参数顺序要看清楚
- 默认参数通常放在后面
- 局部变量不能直接在函数外使用
- 测试时要看边界：499、500；399、400
- 小函数负责具体规则，主函数负责组织流程
- 学会函数，不只是会写代码，更是会“封装规则”。

把业务规则写成函数，你的代码会更清晰、更易维护！



记住：函数不是背语法，而是把重复业务规则写成可复用、可测试、可修改的代码。

业务问题：多笔订单使用同一结算规则

```
orders = [
{"id": "0001", "unit_price": 120, "quantity": 3},
{"id": "0002", "unit_price": 80,  "quantity": 8},
{"id": "0003", "unit_price": 260, "quantity": 2}
]
```

订单结算规则

- 商品金额等于商品单价乘以购买数量；
- 商品金额不低于500元时，享受九折优惠；
- 优惠后金额不低于400元时免运费，否则收取20元运费；
- 实付金额等于优惠后金额加上运费。

需要完成的任务

对每笔订单执行相同的结算规则，输出订单号、商品金额、优惠金额、运费和实付金额。

思考：如果需要处理100笔订单，或者促销规则发生变化，怎样避免反复编写和修改相同的计算代码？

循环可以重复处理数据

```
for order in orders:
  subtotal = order["unit_price"] * order["quantity"]
  discount = 0

  if subtotal >= 500:
    discount = subtotal * 0.10

  after_discount = subtotal - discount
  shipping = 20
  if after_discount >= 400:
    shipping = 0
  payable = after_discount + shipping
  print(order["id"], payable)
```

循环没有解决全部问题

- 循环可以让多笔订单依次使用这段代码；
- 订单预览、正式结算和报表生成都可能需要相同规则（多个场景要用）；
- 如果复制这段代码，规则变化时就需要修改和检查多个位置。

函数可以复用业务规则

```python
def calculate_payable(unit_price, quantity):
    subtotal = unit_price * quantity
    discount = 0
    if subtotal >= 500:
        discount = subtotal * 0.10
    after_discount = subtotal - discount
    shipping = 20
    if after_discount >= 400:
        shipping = 0
    return after_discount + shipping

print(calculate_payable(120, 3))  # 380.0
print(calculate_payable(80, 8))   # 576.0
print(calculate_payable(260, 2))  # 468.0
```

封装后的变化

- `calculate_payable` 为订单结算规则提供了清晰的名称；
- 每次调用只需提供不同的商品单价和购买数量；
- 结算规则只定义一次，规则变化时只需修改函数内部的代码。

函数定义的基本结构

```
def calculate_subtotal(unit_price, quantity):
  subtotal = unit_price * quantity
return subtotal
```

函数的组成部分

- `def` 表示开始定义一个函数；
- `calculate_subtotal` 是函数名，用来说明函数的任务；
- `unit_price` 和 `quantity` 接收调用时传入的数据；
- 缩进代码完成计算，`return` 返回计算结果。

无参数函数

函数可以没有参数，但圆括号不能省略。例如：`def show_report():`

一次函数调用的执行过程

```
def calculate_subtotal(unit_price, quantity):
  subtotal = unit_price * quantity
return subtotal

order_subtotal = calculate_subtotal(120, 3)
print(order_subtotal)  # 360
```

执行过程

1. Python读取函数定义，但暂时不执行函数体；
2. 程序调用函数，把 `120` 和 `3` 传入函数；
3. 函数计算商品金额，并通过 `return` 返回 `360`；
4. 返回值赋给 `order_subtotal`，程序继续向下执行。

参数与传入的值

定义函数时，`unit_price` 和 `quantity` 是参数；调用函数时，`120` 和 `3` 是实际传入的值。

返回值、输出与None

```python
def calculate_shipping(payable):
  if payable >= 400:
    return 0
  return 20

shipping = calculate_shipping(380)
total = 380 + shipping
print(total)  # 400
```

```python
def show_shipping(shipping):
  print(f"运费: {shipping}元")

result = show_shipping(20)
print(result)  # None
```

return的作用

`return` 把结果交给调用者，并立即结束当前函数调用。返回值可以保存到变量中，也可以继续参与计算。函数没有执行 `return` 时，默认返回 `None` 。

位置参数与关键字参数

```python
def calculate_discount(amount, discount_rate):
  return amount * discount_rate

# 位置参数：按照位置匹配
discount_1 = calculate_discount(1000, 0.10)

# 关键字参数：按照参数名匹配
discount_2 = calculate_discount(discount_rate=0.10, amount=1000)

# 混合调用：位置参数必须写在关键字参数前面
discount_3 = calculate_discount(1000, discount_rate=0.10)
```

三种调用方式

- 位置参数按照函数定义中的参数顺序进行匹配；
- 关键字参数通过参数名匹配，顺序可以调整；
- 混合使用时，位置参数必须放在关键字参数前面。

选择建议：参数较少且含义明显时，可以使用位置参数。参数较多或数值含义容易混淆时，关键字参数更容易阅读和检查。

默认参数与函数接口

```
def calculate_shipping(
payable,
free_shipping_threshold=400,
standard_fee=20
):
  if payable >=
  ↪  free_shipping_threshold:
    return 0
  return standard_fee

# 使用默认的满400元免运费规则
fee_1 = calculate_shipping(380)  # 20

# 临时使用满300元免运费规则
fee_2 = calculate_shipping(
380, free_shipping_threshold=300
)  # 0
```

默认参数如何工作

调用时没有提供参数值，函数就使用定义中的默认值。

调用者也可以通过关键字参数覆盖默认值。

参数设计原则

- 必须提供的参数写在前面，有默认值的参数写在后面；
- 默认值应表示常用、稳定且容易理解的业务规则；
- 覆盖默认值时，优先使用关键字参数说明数值的业务含义。

局部变量的作用范围

```
def calculate_subtotal(unit_price, quantity):
  subtotal = unit_price * quantity
  return subtotal

order_subtotal = calculate_subtotal(120, 3)

print(order_subtotal)  # 360
print(subtotal)        # NameError
```

函数内部的变量

- 参数和函数内部定义的变量属于当前函数调用；
- `subtotal` 只能在函数内部使用；
- 函数外部直接访问 `subtotal` 会产生 `NameError` 。

返回值连接函数内外

函数通过 `return` 返回计算结果。函数外部使用新的变量 `order_subtotal` 接收并继续使用这个结果。

让数据通过参数进入函数

```
discount_rate = 0.10

def calculate_discount(amount):
  return amount * discount_rate
```

隐藏的外部依赖

函数签名中没有出现折扣率，但计算结果依赖外部变量 `discount_rate`。阅读和测试函数时容易忽略这个条件。

```
def calculate_discount(amount, discount_rate):
  return amount * discount_rate

discount = calculate_discount(1000, 0.10)
```

清晰的数据流

调用者通过参数提供金额和折扣率，函数通过返回值交回结果。函数需要的数据和产生的结果都可以直接从代码中看出。

清晰的函数接口

```python
def calculate_discount(
amount: float,
discount_rate: float
) -> float:
"""根据订单金额和折扣率计算优惠金额。"""
  return amount * discount_rate
```

好函数的基本特征

- 一个函数只完成一项清晰的任务；
- 函数名和参数名能够说明业务含义；
- 函数需要的数据通过参数传入；
- 函数通过返回值提供稳定、可继续使用的结果。

类型提示与文档字符串

类型提示说明参数和返回值的预期类型，文档字符串说明函数的用途。它们有助于阅读和检查代码，但Python不会仅凭类型提示自动拒绝错误类型。

用测试验证业务规则

```
def calculate_shipping(payable: float, threshold: float = 400,
↪  standard_fee: float = 20) -> float:
"""计算订单运费。"""
  if payable >= threshold:
    return 0
  return standard_fee

assert calculate_shipping(200) == 20
assert calculate_shipping(399) == 20
assert calculate_shipping(400) == 0
assert calculate_shipping(800) == 0
assert calculate_shipping(380, threshold=300) == 0
```

测试覆盖的情况

- 普通金额用于检查基本计算；
- 阈值前后的金额用于检查边界条件；
- 修改免运费门槛用于检查参数是否真正生效。

订单结算任务

```
order = {"id": "0002", "unit_price": 80, "quantity": 8}
```

结算规则

- 商品金额不低于500元时，优惠比例为10%；
- 优惠后金额不低于400元时免运费；
- 普通运费为20元。

任务分解

- `calculate_subtotal()` 计算商品金额；
- `calculate_discount()` 计算优惠金额；
- `calculate_shipping()` 计算运费；
- `settle_order()` 组合前面的函数并生成结算结果。

预期结果商品金额为640元，优惠金额为64元，运费为0元，实付金额为576元。

分别实现结算规则

```python
def calculate_subtotal(unit_price, quantity):
  return unit_price * quantity

def calculate_discount(subtotal, threshold=500, rate=0.10):
  if subtotal >= threshold:
    return subtotal * rate
  return 0

def calculate_shipping(after_discount, threshold=400, standard_fee=20):
  if after_discount >= threshold:
    return 0
  return standard_fee

assert calculate_subtotal(80, 8) == 640
assert calculate_discount(640) == 64
assert calculate_shipping(576) == 0
```

小函数的作用

每个函数只处理一条业务规则，可以单独阅读、运行和测试。某条规则发生变化时，只需要修改对应的函数。

组合函数生成结算结果

```
def settle_order(order):
    subtotal = calculate_subtotal(
    order["unit_price"], order["quantity"]
    )
    discount =
    ↪  calculate_discount(subtotal)
    after_discount = subtotal - discount
    shipping =
    ↪  calculate_shipping(after_discount)
    payable = after_discount + shipping
    return {"id": order["id"], "subtotal":
    ↪  subtotal, "discount": discount,
    ↪  "shipping": shipping, "payable":
    ↪  payable}

result = settle_order(order)
print(result)
assert result["subtotal"] == 640
assert result["payable"] == 576
```

函数之间的数据流

商品金额成为优惠函数的输入，优惠后金额成为运费函数的输入，主函数最后把各项计算结果整理为一个字典。

组合函数的意义

- 小函数负责具体规则，主函数负责组织业务流程；
- 每一步都有清晰的输入和输出；
- AI智能体生成或修改代码后，可以逐个函数进行核验。

本讲总结

函数解决的问题

函数为一项业务任务命名，并把相关代码集中在一个位置。调用者只需要提供数据，不需要重复编写处理过程。

函数的数据流

- 参数说明函数需要哪些输入；
- 局部变量保存函数内部的中间结果；
- 返回值把计算结果交给调用者继续使用。

判断函数质量

- 函数是否只完成一项清晰的任务？
- 函数名、参数和返回值是否容易理解？
- 正常值和边界值测试是否都能通过？

课后实践：批量订单结算

实践任务

沿用本讲开始时的 `orders` 列表和综合实践中的结算函数，生成全部订单的结算结果。

功能要求

- 使用循环调用 `settle_order()`；
- 把每笔订单的结算结果保存到列表；
- 输出订单编号、商品金额、优惠金额、运费和实付金额；
- 计算全部订单的实付金额合计。

验收结果

三笔订单的实付金额应分别为380元、576元和468元，实付金额合计应为1424元。

测试要求

增加商品金额为499元和500元的订单，检查折扣边界；再设计优惠后金额接近400元的订单，检查运费边界。

课后拓展：全局变量

```python
discount_rate = 0.10

def calculate_discount(amount):
# 读取全局变量不需要global声明
  return amount * discount_rate

def update_discount_rate(new_rate):
# 为全局变量重新赋值需要global声明
  global discount_rate
  discount_rate = new_rate

print(calculate_discount(1000))  # 100.0

update_discount_rate(0.15)
print(calculate_discount(1000))  # 150.0
```

关于全局变量

函数可以读取全局变量。在函数内部为全局变量重新赋值时，需要使用 `global` 声明。初学阶段优先使用参数和返回值，减少隐藏依赖。

课后拓展：使用断言检查结果

```python
def calculate_shipping(payable, threshold=400):
  if payable >= threshold:
    return 0
  return 20

assert calculate_shipping(399) == 20
assert calculate_shipping(400) == 0

# 去掉下一行开头的#，观察AssertionError
# assert calculate_shipping(400) == 20, "边界测试失败"
```

assert如何工作

条件为`True`时，程序继续执行且通常没有输出。条件为`False`时，Python产生`AssertionError`。可以在判断条件后添加说明文字。断言失败时，这段文字能够帮助定位不符合预期的业务规则。

使用范围：断言适合开发阶段的测试和内部条件检查，不应代替面向用户的输入验证和错误提示。

课后拓展：不定长参数

```python
def calculate_total(*amounts):
  print(type(amounts))  # tuple
  return sum(amounts)

total = calculate_total(299, 680, 1500)
print(total)  # 2479

def show_order(**order):
  print(type(order))  # dict
  for key, value in order.items():
    print(key, value)

show_order(id="0001", amount=299, status="PAID")
```

两种不定长参数

- `*amounts` 把多个位置参数收集为元组；
- `**order` 把多个关键字参数收集为字典；
- 只有参数数量确实不固定时，才需要使用这种写法。

课后拓展：匿名函数与排序

```
orders = [
{"id": "0001", "amount": 299},
{"id": "0002", "amount": 680},
{"id": "0003", "amount": 1500}
]

sorted_orders = sorted(
orders,
key=lambda order: order["amount"],
reverse=True
)

for order in sorted_orders:
  print(order["id"], order["amount"])
```

匿名函数的作用

`lambda` 创建一个只包含单个表达式的简短函数。本例用它告诉 `sorted()` 按照订单金额排序。

课后拓展：递归处理层级数据

```
category = {
  "name": "商品",
  "children": [
  {"name": "服装", "children": []},
  {
    "name": "数码",
    "children": [
    {"name": "手机", "children": []}
    ]
  }
  ]
}

def show_categories(node, level=0):
  print("  " * level + node["name"])

for child in node["children"]:
  show_categories(child, level + 1)

show_categories(category)
```

递归的基本思想

函数在处理当前分类后，再调用自身处理下一层分类。

每一层数据具有相同的结构，因此可以使用同一段代码。

递归如何结束

当分类的 `children` 为空时，循环不会执行，函数不再调用自身，递归自然结束。

THE END